

Block Semantic Operators for Kalypso: Evaluating Many Tuples per LLM Call in Pipelined Semantic Queries

Shivansh Chhawri, Yash Pathak, Romasha Guin,
Justin Ly, Shri Venkatakrishnan Vasudevan

CS692S Systems for Machine Learning, Prof. Marco Serafini, Fall 2026

Abstract

Semantic query processing systems evaluate LLM-powered operators such as filters, maps, and joins one tuple, or one tuple pair, per LLM request. They rely on the inference engine to batch those requests. We propose adding block operators to Kalypso: a block operator evaluates many tuples in a single LLM call and returns one validated answer per row, or a list of matching pairs for a join.

We will implement block filter, join and map operators and run them as a concurrent pipeline. We will study the input/output-size mismatch created by selective filters and expanding joins, compare fixed, always-reformed, and threshold-based block management, investigate block sizing and tuple ordering for KV-cache reuse, and add recovery for malformed, incomplete, or truncated outputs. Finally, we will compare tuple-at-a-time execution with vLLM batching and block execution across MEDEC and ContractNLI, with BioDEX if time permits, and compare measured block-size optima with Trummer’s cost model.

1 Technical Project Description

1.1 What general problem does the project address?

Systems such as LOTUS [1] and Kalypso [2] treat LLM calls as operators inside a relational query plan. A semantic filter decides whether each tuple satisfies a natural-language predicate. A semantic join decides whether tuples from two relations correspond under a semantic condition. In the above cases, LLM inference is the dominant cost for processing the queries.

The standard execution model issues one LLM request per tuple for filters and maps, and one per candidate pair for joins. Inference engines such as vLLM batch concurrent requests efficiently on the GPU. However, each request is still a separate logical invocation. It repeats the operator instruction, any few-shot examples, and, for joins, one side of the pair.

The project builds on this idea to test the impact of input size for the LLM invocation. In cases where block execution outperforms single-tuple execution, we will study the cases and operators where blocks perform

better, together with the formation and management of blocks as they flow through a pipeline.

Concretely, we address two questions:

1. When does evaluating multiple tuples in one LLM call improve performance relative to independent requests batched by vLLM?
2. How should blocks be managed through pipelines whose filters shrink the tuple stream and whose joins expand it?

The idea resembles vectorized execution in conventional databases, where operators exchange vectors of tuples instead of single tuples [4]. Block operators go further, because the whole block becomes the input of one model invocation. Correctness therefore depends on the model returning a separate, correct answer for every row.

1.2 Limitations of existing solutions

Operator-at-a-time systems. LOTUS [1] defines semantic operators and optimizes them, for example with model cascades. Execution proceeds one operator at a time, and intermediate results are materialized between operators. Each operator invocation on a tuple or pair is an independent LLM request.

Kalypso today. Kalypso [2] is a relational LLM serving layer built on vLLM. A client submits a query plan and an input table. Kalypso groups consecutive tuple-at-a-time operators into stages. It runs each tuple through a stage as a task and pipelines tasks across stages. Its memory-aware scheduler splits the KV-cache budget between stages and moves budget toward backed-up stages. Its prompt layout places the tuple’s content before the operator instruction. This lets the engine’s prefix cache reuse a tuple’s KV state across consecutive operators.

The unit of work, however, is always one tuple. A join fans each left tuple out into one (tuple, candidate) task per candidate, and each task issues its own request. Prefix caching can avoid recomputing a shared left-tuple prefix. Even so, every pair still costs a separate request carrying the join instruction and candidate tuple, and the number of requests grows with the number of candidate pairs. Kalypso’s aggregation operator already packs many tuples into one prompt, but it does not need to

produce a separate reliable answer for each row, so it does not provide the block-operator abstraction required here.

Block-based semantic joins. Trummer [3] proposes placing batches from both relations in one prompt and gives a cost model for choosing the optimal number of tuples from each side. The model accounts for prompt size, tuple lengths, expected output size, join selectivity and the context window.

That model targets token cost and is primarily an operator-level model. In a locally served setting, minimizing tokens need not minimize execution time: prefill and decode have different costs, request size affects GPU utilization and batching, and several block invocations may run concurrently. The model also does not capture pipeline effects. A join block that is efficient in isolation may delay output to a downstream operator.

Vectorized execution. Vectorized systems [4] exchange blocks between operators. Each tuple is still processed by deterministic code, so there is no question of one tuple’s answer being dropped or influencing another’s. With block-level LLM execution, this correctness boundary is no longer automatic.

Reuse versus compaction. Blocking introduces a trade-off absent from tuple-at-a-time execution. Keeping original block boundaries preserves physical layout and potential prefix reuse but carries filtered-out rows, whereas reforming a smaller block removes dead rows but changes the sequence and may discard reusable state.

1.3 Proposed approach

Block abstraction and interface. A block is an ordered list of tuples, each with a stable row ID. A block task is one LLM request that evaluates one operator over one block. In the current scope of the project, block size is set as a query parameter, given either in the form of row counts or a token budget; the experimental sweeps use row counts.

Our preliminary reading of Kalypso’s scheduler identified a natural integration point: the place where waiting tuples are admitted. There, several waiting tuples can be grouped into one block task with one combined memory reservation. Per-row results are fanned back out to downstream stages afterwards. Operators will declare block support in their execution contract alongside Kalypso’s existing properties (pipelining, predicate, Cartesian product).

Block semantic filter. One prompt contains the shared system context, the rows tagged with their IDs, the predicate, and an output specification. The model returns a structured result mapping each row ID to a Boolean.

Block semantic map. One prompt produces a structured per-row output keyed by row ID. This replaces the maps in existing workload plans and amortizes fixed

instruction and schema tokens across many tuples.

Block-size model for filters. A simple token constraint, viewed as a modified version of Trummer’s rule for a single relation, provides a starting point for filter sizing. Let p be the fixed instruction cost, s the average row-content size, s_{verdict} the verdict size, t the context budget, and B the block size. The estimated tokens per call are

$$T_{\text{filter}}(B) = p + Bs + Bs_{\text{verdict}} = p + B(s + s_{\text{verdict}}). \quad (1)$$

The context-window constraint is

$$p + B(s + s_{\text{verdict}}) \leq t, \quad (2)$$

$$B \leq \frac{t - p}{s + s_{\text{verdict}}}, \quad (3)$$

giving the initial analytical block size

$$B^* = \left\lfloor \frac{t - p}{s + s_{\text{verdict}}} \right\rfloor. \quad (4)$$

Measured latency and overflow behavior will determine whether this bound is useful in practice. For joins, we will use Trummer’s formulas for selecting the initial values of b_1^* and b_2^* [3], and compare their predictions with measured optima under local vLLM execution.

Block semantic join. We will implement two shapes:

- $1 \times j$: one left tuple against a block of j candidates. The model returns the IDs of the matching candidates.
- $k \times j$: a block of k left tuples against a block of j right tuples. The model returns the matching pairs.

The $1 \times j$ shape fits ContractNLI naturally, where each contract is paired with a small fixed set of hypotheses. It also fits index-assisted joins, where the retrieved candidate list for a left tuple forms a ready-made candidate block. The $k \times j$ shape reduces the number of requests further, but the model faces more possible pairings within one invocation.

Output validation and recovery. Every block response is parsed and checked for malformed output, missing row IDs, duplicate row IDs, and invented row IDs. Structured or guided decoding will constrain the response schema where supported; otherwise we will use a strict format. Join responses will also include an explicit completion marker so that truncation can be distinguished from a complete result. Rows or pairs without a valid answer are re-asked as a smaller block or evaluated with per-tuple requests. Recovery requests count toward all cost metrics so that the comparison with the baseline remains fair.

Pipelined execution and block management. Operators consume and produce streams of blocks, and downstream stages start as soon as blocks are available. A partially filled block is flushed when either the number of buffered rows reaches the target size, `batch.size`,

or the elapsed time since the previous flush reaches `linger.ms`; any remaining rows are also flushed at end of input. Thus, dispatch is triggered by whichever condition occurs first, trading some request amortization for bounded pipeline latency when a timeout produces an underfilled block.

We will compare three block-management policies:

- *Fixed boundaries.* A block keeps its rows across operators. Its prompt can become a reusable KV prefix for the next operator, but after a selective filter it carries dropped rows and is underfilled.
- *Always re-form.* Surviving rows are regrouped into dense blocks between operators, which creates a new prefill and loses the original prefix.
- *Threshold re-forming.* Rows are re-formed only when occupancy drops below a threshold θ ; otherwise the original block is preserved.

For the re-forming policies, the target block size depends on the downstream operator. A filter-to-filter transition uses the filter-sizing formula above, whereas a filter-to-join transition uses the block sizes predicted by Trummer’s join model [3]. The filter $\rightarrow$ join $\rightarrow$ filter plan exercises all three cardinality effects: the first filter shrinks the stream, the join expands it, and the second filter shrinks it again.

Maximizing KV-cache reuse. In tuple-at-a-time execution, a semantic filter may use a prompt of the form

$$[\text{SYS} + \text{tuple} + \text{FILTER}], \quad (5)$$

whereas a subsequent map over a surviving tuple may use

$$[\text{SYS} + \text{tuple} + \text{MAP}]. \quad (6)$$

The shared prefix $[\text{SYS} + \text{tuple}]$ creates an opportunity for a KV-cache hit. With block execution, however, filtering rows from a block or changing their order before the next operator can alter the prompt prefix and reduce reuse.

As a possible solution, we will investigate survival-aware tuple ordering. A lightweight model or a retrieval-based method such as BM25 will estimate the probability P_i that tuple i survives the semantic filter. Let L_i denote the tuple’s token length. Before a block is passed to a stage, its tuples will be ordered in descending order of

$$\frac{L_i}{P_i}. \quad (7)$$

This heuristic combines tuple length with predicted survival probability with the goal of preserving a larger reusable prefix across consecutive operators. We will evaluate whether this ordering improves cache reuse enough to offset its overhead.

Preliminary work (feasibility only). Kalypso runs locally behind vLLM with a small open model (Llama 3.2 3B), and the benchmark queries run and can be scored against dataset labels. A 10-row MEDEC run completed

end-to-end, with about 60% of prompt tokens served from the prefix cache. This confirms the per-tuple KV reuse that fixed block boundaries aim to preserve. No block operator has been implemented or measured yet.

Design questions the project will investigate:

1. Block formation: fixed size or token budget, and how long to wait before running a partial block.
2. Block boundaries through the pipeline: fixed, re-formed, or re-formed at an occupancy threshold.
3. Join shapes and output formats: $1 \times j$ versus $k \times j$, reliable match-list formats, and combination with index-assisted candidates.
4. The output contract and the cost of recovery.
5. Memory: the balance between larger reservations and fewer concurrent requests.
6. Pipeline overlap: how per-operator block sizes affect when downstream operators start.
7. Cost model: how well Trummer’s recommended block sizes match measured latency under vLLM.

1.4 Implementation challenges: what is hard?

Reliable per-row answers. As blocks grow, a small model may omit rows, repeat or invent IDs, mix information between rows, or let one row bias another’s decision. In a large candidate block it may also miss join matches. Output handling must be robust, and recovery must not quietly erase the benefit of blocking.

Context and output sizing. Join input and output sizes grow with the block dimensions and match density. Oversized blocks can exceed the context window or produce long decode times. The system must detect overflow and retry using smaller blocks.

Prefix reuse versus occupancy. Keeping block boundaries fixed preserves the KV prefix for the next operator but leaves blocks underfilled after selective filters. Re-forming fills blocks but forces a fresh prefill. The better policy likely depends on selectivity, row length and block size, and we have no a priori answer.

Memory estimation for block tasks. Kalypso admits a task only if its estimated memory fits its stage’s budget. For output length, it relies on an online estimate per operator and retries a task with a larger bound if the estimate is exceeded [2]. Block tasks have much larger prompts, and their output size scales with the number of rows or matches. A block join’s output depends on how many pairs match, which is unknown in advance. An underestimate forces a whole block to be retried; an overestimate reduces how many block tasks run concurrently.

Latency versus occupancy. Waiting to fill a block delays downstream operators, while running partial blocks wastes the per-request savings. This tension is sharpest behind a selective filter or a low-fanout join. The

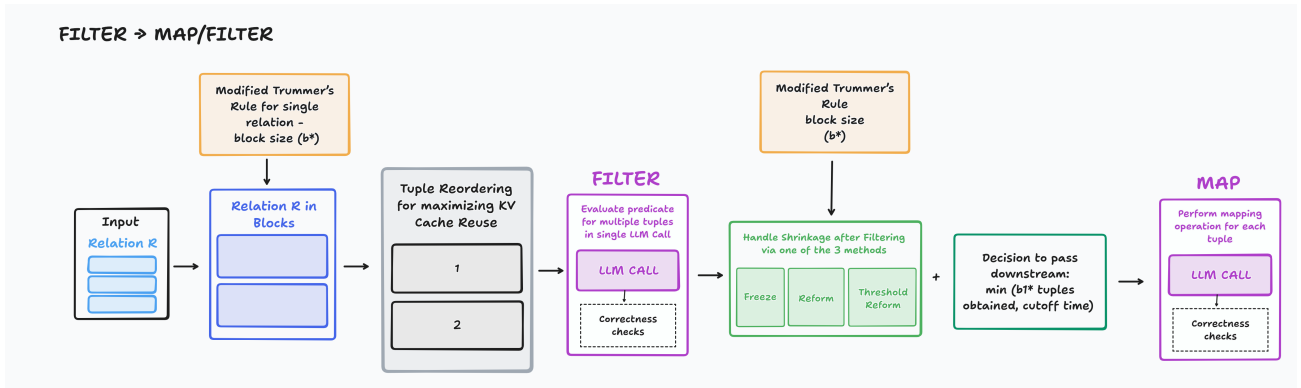


Figure 1: Proposed block pipeline for a filter followed by a map or another filter.

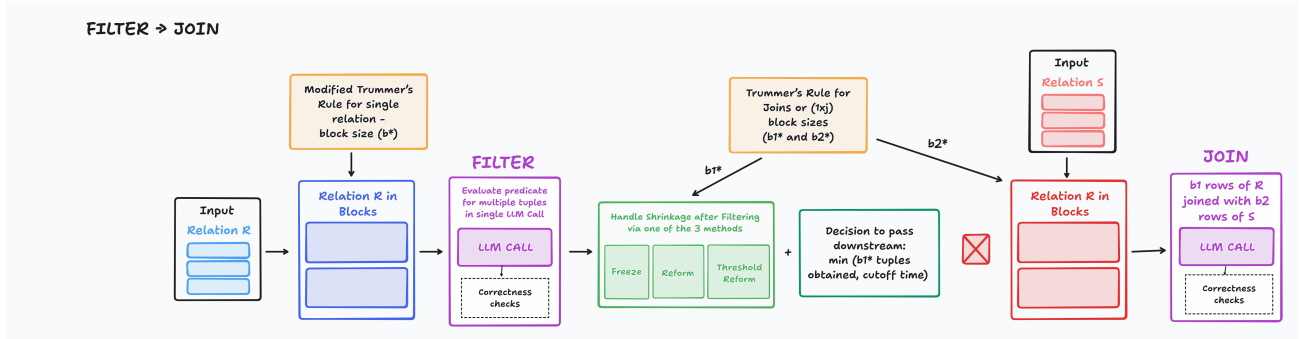


Figure 2: Proposed block pipeline for a filter followed by a semantic join.

`batch.size` and `linger.ms` parameters will therefore be measured explicitly.

Context window. ContractNLI contracts average about 2,100 tokens [2]. Blocks with contracts on the left side therefore grow quickly, which limits k and makes prefill time large per block.

Scheduling and measurement. Kalypso prioritizes downstream tasks. The engine honors these priorities only when configured for priority scheduling, which may affect pipeline overlap. Demonstrating that all three operators are active concurrently also requires per-operator activity tracing.

Scale. Experiments run on a single local machine with a small model, so batching behavior differs from a multi-GPU server. We treat results as a proof of concept and focus on relative comparisons.

1.5 Evaluation criteria: how will project success be measured?

Workloads (from Kalypso’s benchmark suite):

- *ContractNLI (primary)*. Contracts $\times$ hypotheses. We will replace the plan’s final map with a filter to obtain filter $\rightarrow$ join $\rightarrow$ filter.
- *MEDEC*. Filter $\rightarrow$ map $\rightarrow$ map, for block filter and block map.
- *BioDEX (if time permits)*. Its index-assisted join

supplies natural $1 \times j$ candidate blocks.

We will test the proof of concept on a subset of these datasets selected according to available computational resources.

Configurations:

- (a) Kalypso today: tuple-at-a-time with vLLM batching.
- (b) Block operators, materialized: each operator finishes before the next starts.
- (c) Block operators, pipelined, with fixed block boundaries.
- (d) Block operators, pipelined, with block re-forming (always and threshold-based).

Variables:

- block size (1, 2, 4, 8, 16, 32, 64; k and j for joins);
- row length and prompt length;
- filter selectivity, controlled through predicate choice and input subsets;
- join fanout, controlled through candidate count or index top- k .

Metrics. The required metrics are end-to-end latency, number of LLM requests, semantic accuracy and peak memory. As diagnostics we will also record: throughput; prefilled and generated tokens; prefix-cache hit rate; block occupancy after each operator; time until each downstream operator emits its first output; block failure

Table 1: Work assignments per milestone for each group member.

Member	M1 (Oct 7)	M2 (Oct 21)	M3 (Nov 4)	M4 (Nov 18)
Shivansh	Block task grouping at scheduler admission; block filter prototype; validator design	Block join and map; recovery; memory-reservation integration	Block streaming between stages; re-forming policies; integration, debugging and demo	Experiment design; performance and cost-model analysis; core report sections
Yash	Measurement harness script; baseline runs and table	Harness extended with occupancy, failure and recovery metrics; automated sweep runner	Per-operator activity logging; run matrix for (a)–(d)	Full sweep runs; results archive
Venkat	Filter $\rightarrow$ join $\rightarrow$ filter plan for ContractNLI; sample subsets and dataset statistics	Input subsets with controlled selectivity (low, medium, high) and controlled fanout	Output completeness checks across configurations; demo dataset	MEDEC (and BioDEX if time permits) runs
Romasha	Block filter prompt template; unit tests for the output parser on hand-written malformed outputs	Accuracy scorer: labels and agreement with tuple-at-a-time answers, per block size	Accuracy results for (a)–(d)	Accuracy-versus-block-size analysis; counts of failure types
Justin	One-page summary of Trummer’s model and its inputs; plotting script for the baseline table	Cost-model calculator that takes measured token statistics and outputs predicted (k, j) ; first ContractNLI predictions	Overlap timeline and per-operator occupancy plots	Final figures; cost-model prediction table

and recovery rates.

Accuracy is treated as a relative measure: does it degrade as blocks grow? We report it against dataset labels where they exist. We also report agreement with the same model’s tuple-at-a-time answers, because the small model’s absolute accuracy is not the point.

Cost-model study. For each workload, we will compute the block sizes recommended by Trummer’s model. We will then measure the empirically best block size for the join in isolation and for the full filter $\rightarrow$ join $\rightarrow$ filter pipeline. Finally, we will analyze which assumptions hold under vLLM and where predictions diverge. If discrepancies are clear and systematic, a simple adjusted heuristic is a stretch goal; we do not plan a general optimizer.

The project succeeds if:

1. Block filter, join and map run inside Kalypso with one LLM call per block, and every query returns a complete result, with invalid block outputs detected and recovered.
2. A traced run of filter $\rightarrow$ join $\rightarrow$ filter shows all three operators active concurrently.
3. Configurations (a)–(d) are compared on the required metrics across the variable sweeps.
4. We can state, with measurements, when blocking helps, when re-forming helps, and how accuracy changes with block size.
5. We deliver a predicted-versus-measured block-size comparison for Trummer’s model, with an explanation of the gaps.

2 Milestone Plan

Roles. Shivansh Chhawri: core engine design and implementation, experiment design and analysis. Yash: baseline runs and the measurement harness. Venkat: workload plans and datasets. Romasha: prompt templates and accuracy scoring. Justin: cost-model calculator and figures. All members contribute to the report.

Milestone 1 (Oct 7): Baseline and block filter prototype.

- One command runs unmodified Kalypso on sampled MEDEC and ContractNLI and writes a metrics table (latency, requests, prompt and generated tokens, prefix-cache hit rate, peak KV memory).
- The ContractNLI filter $\rightarrow$ join $\rightarrow$ filter plan runs in unmodified Kalypso.
- A block filter prototype evaluates MEDEC’s filter with block sizes 1, 4, and 16 in one call per block. Its output is validated, and it returns a result for every row.

Milestone 2 (Oct 21): Block join, block map, recovery.

- Block join ($1 \times j$ and $k \times j$) with match-list output, and block map.
- A recovery path for missing, duplicate and invented IDs and malformed output, with recovery counts logged.
- Block tasks reserve memory through Kalypso’s memory manager.
- A table of isolated-operator sweeps: block filter at block sizes 1–32 on MEDEC, and block join over j on ContractNLI.

Milestone 3 (Nov 4): Fully functioning prototype.

- Pipelined filter $\rightarrow$ join $\rightarrow$ filter with block operators, sup-

porting fixed boundaries, always re-forming and threshold re-forming, plus a flush timeout.

- An activity trace showing that the join starts before the first filter finishes, and that the second filter starts before the join finishes.
- End-to-end runs of configurations (a)–(d) on the ContractNLI plan at one or more block settings, reporting all four required metrics with complete outputs.

Milestone 4 (Nov 18): Experimental study and cost-model evaluation.

- Sweeps over block size, selectivity and fanout for configurations (b)–(d), compared with (a).
- An analysis of when re-forming beats fixed boundaries.
- A predicted-versus-measured table of best block sizes, for the isolated join and for the pipeline, with a list of the model assumptions that hold and those that diverge.
- A draft of the final report. Stretch: an adjusted block-size heuristic, if the gaps are systematic.

Work assignments for each milestone and group member are listed in Table 1.

References

- [1] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *PVLDB* 18(11), 2025.
- [2] H. Son, M. A. Islam, H. G. Cao, H. Guan, and M. Serafini. Kalypso: Relational LLM Serving. arXiv:2607.23815, 2026.
- [3] I. Trummer. Implementing Semantic Join Operators Efficiently. arXiv:2510.08489, 2025.
- [4] P. Boncz, M. Zukowski, and N. Nes. MonetDB/X100: Hyper-Pipelining Query Execution. In *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2005.